

Realizzazione di una applicazione OCR per manoscritti italiani mediante Kraken

Francesco Rombaldoni¹

¹f.rombaldoni@campus.uniurb.it

Riassunto

Il lavoro svolto per l'esame consiste nel *fine tuning* di un modello pre-esistente di Kraken, con l'obiettivo di ottenere un sistema in grado di leggere manoscritti in lingua italiana. Nel contesto in cui lavoro (pubblica amministrazione) esiste infatti una forte necessità di strumenti OCR avanzati per la digitalizzazione di documenti storici che, a causa delle normative sulla privacy, dovranno essere distrutti dopo la scadenza dei tempi di conservazione.

Per raggiungere questo obiettivo sono stati scelti alcuni documenti di cui si possedevano già le trascrizioni. Le pagine sono state scannerizzate, pre-elaborate con strumenti standard (in particolare ScanTailor) e quindi utilizzate per generare il *ground truth* necessario all'addestramento. I dati così ottenuti sono stati organizzati e normalizzati mediante una serie di script in Python, sviluppati appositamente per l'utilizzo con Kraken.

Il risultato di questo processo è un modello specializzato per la calligrafia italiana adottata nei documenti di riferimento, che raggiunge un'accuratezza media di circa l'88% a livello di parola su pagine simili a quelle di addestramento (con una deviazione standard di circa 2 punti percentuali). Sebbene il modello non sia ancora sufficientemente affidabile per un utilizzo completamente automatico sulla documentazione clinica, esso costituisce una prova di concetto concreta e già utile per motivare la richiesta di risorse aggiuntive a livello di comparto, in vista dello sviluppo di uno strumento OCR conforme alle esigenze normative e organizzative dell'INRCA.

1 Introduzione

1.1 Il problema

Il problema affrontato in questo lavoro nasce dall'esigenza di adeguare la gestione documentale dell'INRCA alle normative vigenti in materia di protezione dei dati personali e conservazione dei documenti, in particolare il Regolamento (UE) 2016/679 (GDPR) e la norma ISO/IEC 21964, che disciplina la transizione dal supporto cartaceo al digitale.

L'INRCA, in quanto istituto sia ospedaliero sia di ricerca, è tenuto a garantire il rispetto di tali obblighi, in particolare per quanto riguarda la catalogazione, l'archiviazione e la successiva distruzione dei documenti sensibili. Ciò rende necessario disporre di strumenti OCR affidabili, in grado di supportare la digitalizzazione massiva di documenti storici, spesso contenenti anche parti manoscritte.

Una prima analisi ha preso in considerazione alcuni

software commerciali largamente diffusi:

- Wondershare PDFelement;
- ABBYY FineReader PDF 16;
- Nanonets;
- Adobe Acrobat.

Tali strumenti sono stati tuttavia scartati in quanto inadeguati al caso d'uso specifico. In particolare, nessuno di essi offre un supporto realmente efficace per documenti contenenti porzioni manoscritte, come ricette mediche o note su pazienti, che costituiscono una parte rilevante della documentazione sanitaria.

In linea teorica, gli OCR basati su tecnologie di intelligenza artificiale sarebbero gli unici in grado di affrontare il carico di lavoro descritto. Tuttavia, la maggior parte delle soluzioni commerciali più avanzate richiede la trasmissione remota delle pagine da analizzare verso servizi cloud esterni, il che non consen-

te di garantire un controllo completo sulla sicurezza delle informazioni trattate.

Alla luce di queste considerazioni, la necessità è quella di disporre di un sistema OCR:

- sufficientemente efficiente da poter essere eseguito interamente in locale;
- configurabile e controllabile dagli operatori interni;
- adatto a gestire manoscritti in lingua italiana.

Fra le possibili alternative, la scelta è ricaduta su Kraken, un OCR open source basato su reti neurali profonde, che consente il *fine tuning* di modelli pre-esistenti sulla base di dati di addestramento personalizzati. La documentazione ufficiale di Kraken [2] e il relativo codice sorgente [1] forniscono una descrizione dell'architettura generale e degli strumenti di addestramento, ma le informazioni risultano piuttosto frammentate: parte del materiale è ospitato su repository non più mantenute e parte su archivi istituzionali europei (ad esempio su Zenodo [3]), dove alcune pagine storicamente collegate a Kraken non risultano più disponibili.

1.2 Input e output

Gli *input* del sistema sono le scannerizzazioni dei documenti da digitalizzare, sottoposte preventivamente a un processo di pulizia e normalizzazione tramite ScanTailor (correzione della centratura, ritaglio delle parti non informative, adeguamento del contrasto), mantenendo la risoluzione originale di 600 dpi.

Gli *output* considerati in questo lavoro sono file di testo semplice (.txt) contenenti le trascrizioni delle pagine. Ai fini specifici del progetto, non è stato ritenuto necessario ottenere un PDF in cui il testo sia selezionabile e sovrapposto all'immagine: l'obiettivo è piuttosto quello di creare, per ciascun documento, un "pacchetto" costituito dall'immagine scannerizzata e dalla relativa trascrizione.

Questa organizzazione consente di:

- eseguire ricerche testuali sulle trascrizioni per individuare rapidamente i documenti di interesse;
- visualizzare, in caso di necessità, l'immagine originale della pagina, preservando l'aspetto grafico del documento.

1.3 Architettura del modello: CRNN, CTC e LSTM bidirezionali

Il modello utilizzato da Kraken per il riconoscimento delle righe di testo è una *Convolutional Recurrent Neural Network* (CRNN) addestrata con una funzione di costo di tipo *Connectionist Temporal Classification* (CTC) e dotata di strati ricorrenti LSTM (*Long Short-Term Memory*) bidirezionali.

CRNN: combinazione di convoluzioni e ricorrenza Una CRNN combina due idee principali:

- **Strati convoluzionali (CNN):** estraggono automaticamente caratteristiche locali dall'immagine della riga (contorni, tratti di penna, forme di lettere, ecc.). A differenza di metodi basati su feature ingegnerizzate a mano, la CNN impara direttamente dal dato quali pattern visivi sono utili per discriminare i caratteri.
- **Strati ricorrenti:** modellano la dipendenza sequenziale lungo la riga. Dopo le convoluzioni, l'immagine viene "compressa" in una sequenza di vettori di feature (uno per ciascuna posizione orizzontale). Gli strati ricorrenti elaborano questa sequenza tenendo conto del contesto a sinistra e a destra.

In pratica, l'immagine di una riga viene prima ridotta e trasformata da una CNN in una sequenza $\{\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_T\}$, dove ogni $\mathbf{f}_t$ è un vettore di feature che riassume le informazioni visive di una "finestra" verticale della riga. Questa sequenza di feature viene poi passata a una rete ricorrente che produce, per ogni passo temporale t , una rappresentazione arricchita dal contesto.

LSTM bidirezionali Per modellare le dipendenze a lungo raggio nella sequenza di feature, Kraken utilizza strati ricorrenti di tipo LSTM. Le LSTM sono un'estensione delle RNN classiche progettata per evitare il problema del gradiente che svanisce: introducono celle di memoria e meccanismi di "porte" (input, output, forget) che consentono di mantenere o dimenticare informazioni nel tempo in modo controllato.

Nel caso di Kraken le LSTM sono **bidirezionali**: per ogni posizione t della sequenza, la rete calcola due stati:

- uno in direzione *forward*, che processa la sequenza da sinistra a destra;
- uno in direzione *backward*, che processa la sequenza da destra a sinistra.

Questi due stati vengono concatenati per ottenere una rappresentazione che tiene conto sia del contesto precedente sia di quello successivo rispetto a ciascuna posizione. Questo è particolarmente utile nel riconoscimento di testo manoscritto, dove l'interpretazione di un carattere può dipendere fortemente dalle lettere adiacenti (ad esempio per distinguere tra “m” e “rn”).

Funzione di costo CTC (Connectionist Temporal Classification) L'uscita della parte ricorrente della rete è, per ogni passo temporale t , un vettore di probabilità sui possibili simboli (lettere, numeri, segni di punteggiatura e un simbolo speciale “blank”). Il problema è che:

- non è nota a priori la corrispondenza esatta tra posizioni temporali e caratteri della trascrizione;
- la lunghezza della sequenza di output (numero di passi temporali) è di solito diversa dalla lunghezza della trascrizione in caratteri.

La funzione di costo **CTC** è stata introdotta proprio per gestire questo tipo di situazioni. L'idea di base è:

- la rete produce una sequenza di simboli “estesa”, che può contenere ripetizioni e simboli *blank*;
- una procedura di *collasso* (*collapse*) elimina i simboli *blank* e comprime le ripetizioni consecutive, ottenendo così una sequenza di caratteri “pulita”;
- la CTC definisce tutte le possibili sequenze estese che, dopo il collasso, producono la trascrizione desiderata, e massimizza la probabilità totale di questo insieme di allineamenti.

In altre parole, durante l'addestramento non è necessario fornire un allineamento esplicito “pixel per carattere”: è sufficiente avere, per ogni immagine di riga, la sua trascrizione completa. La CTC si occupa di sommare la probabilità di tutti gli allineamenti temporalmente compatibili con quella trascrizione.

Formalmente, se indichiamo con $\mathbf{y}_t$ la distribuzione di probabilità sui simboli al tempo t e con ℓ la sequenza di caratteri corretta, la CTC definisce:

$$\mathcal{L}_{\text{CTC}} = -\log P(\ell \mid \mathbf{y}_1, \dots, \mathbf{y}_T)$$

dove $P(\ell \mid \mathbf{y}_{1:T})$ è la somma delle probabilità di tutte le sequenze estese che collasano in ℓ . Il calcolo di questa probabilità avviene tramite un algoritmo di tipo *forward-backward*, simile a quello utilizzato nei modelli di Markov nascosti.

Decodifica della sequenza testuale In fase di inferenza, dato un output probabilistico per ogni passo temporale, si può applicare una decodifica CTC semplice (ad esempio prendendo ad ogni passo il simbolo con probabilità massima e poi collassando ripetizioni e *blank*) oppure metodi più sofisticati basati su *beam search* e, se disponibile, modelli linguistici di supporto.

Nel caso considerato, Kraken utilizza la decodifica CTC per ricostruire la sequenza testuale a partire dalle distribuzioni di probabilità prodotte dal modello CRNN. Questo approccio consente di addestrare il modello direttamente su coppie immagine-trascrizione senza richiedere un allineamento manuale carattere-pixel, ed è particolarmente adatto al riconoscimento di testo manoscritto, dove la segmentazione esplicita dei singoli caratteri è spesso ambigua o poco affidabile.

1.4 Il modello

Per l'addestramento è stato utilizzato come punto di partenza il modello `Tridis_Medieval_EarlyModern.mlmodel`, ottenuto tramite il gestore dei modelli incluso in Kraken. La documentazione (oggi parzialmente reperibile solo tramite riferimenti indiretti, ad esempio su archivi come Zenodo [3]) suggerisce che questo modello, addestrato su manoscritti latini tardo-medievali e di età moderna, costituisca una buona base di partenza per il riconoscimento di grafie storiche europee. L'informazione secondo cui tale modello sarebbe adatto anche come base per manoscritti italiani deriva in parte da risposte fornite da strumenti di intelligenza artificiale, che a loro volta facevano riferimento a lavori ospitati su repository Zenodo non più direttamente accessibili.

La scelta di questo modello di partenza è motivata dalla somiglianza geometrica e stilistica tra la grafia dei manoscritti latini più recenti e quella dei manoscritti italiani novecenteschi presenti nel contesto di interesse. L'ipotesi di lavoro è che un modello già addestrato su una grande varietà di manoscritti latini possa costituire una buona base per un *fine tuning* mirato sui manoscritti italiani, richiedendo quindi un numero relativamente contenuto di pagine annotate manualmente.

2 Metodi

Il processo di addestramento ha seguito una metodologia articolata in più fasi, resa più complessa dal fatto che Kraken, originariamente nato come progetto open source, è stato successivamente integrato in iniziative istituzionali europee. Nel tempo, parte della

documentazione è stata spostata o resa meno facilmente accessibile, e molte delle guide online puntano oggi a repository non più mantenute o a pagine che restituiscono errori 404. La documentazione ufficiale disponibile sul sito del progetto [2] e sul repository GitHub [1] fornisce quindi solo una parte del quadro complessivo.

Per orientarmi in questo scenario frammentato ho fatto ampio uso di strumenti di supporto (tra cui GitHub Copilot, reso disponibile dal progetto scolastico di GitHub), al fine di ricostruire una pipeline di addestramento completa e riproducibile.

Dal punto di vista operativo, disponendo di una scheda grafica NVIDIA, ho scelto come sistema operativo Pop_OS, in quanto offre un supporto immediato ai driver proprietari NVIDIA e all'uso dei CUDA core (nel mio caso versione 12). Seguendo le indicazioni raccolte, ho creato un ambiente `conda` dedicato e installato Kraken tramite `pip`, insieme alle dipendenze necessarie.

Tutta la logica di preparazione dei dati e gestione dei file di *ground truth* è stata implementata in Python, con l'obiettivo di ottenere uno strato di astrazione relativamente semplice da riutilizzare in contesti simili.

2.1 Gestione delle immagini

Come base di addestramento sono stati utilizzati circa 150 fogli di documenti, redatti a mano e già trascritti in precedenza. I fogli sono stati scannerizzati utilizzando una stampante da ufficio professionale con risoluzione di 600 dpi.

Le immagini ottenute sono state quindi pre-elaborate con il software open source ScanTailor, utilizzando principalmente le impostazioni standard. In questa fase sono stati corretti:

- la centratura delle pagine;
- il ritaglio dei margini non informativi;
- il bilanciamento complessivo del contenuto visivo, mantenendo la risoluzione a 600 dpi.

Successivamente, ogni pagina è stata suddivisa in singole righe di testo. A ciascuna riga è stata associata la relativa trascrizione manuale, in modo da costruire un dataset di coppie immagine-testo utilizzabile per il *fine tuning*. La struttura dei file è stata organizzata in cartelle dedicate per le immagini e per i testi, con convenzioni di denominazione coerenti per facilitare gli abbinamenti automatici.

2.2 Il processo di addestramento

Generazione del *ground truth* dai file .md riga per riga Le trascrizioni manuali erano conservate in formato Markdown, scelta che rendeva più agevole la lettura umana ma richiedeva una fase di pulizia prima di poter essere utilizzate come *ground truth*. Per questo è stato sviluppato uno script Python che, dato un insieme di immagini e i corrispondenti file .md, genera un file `ground_truth.txt` nel formato atteso da `ketos` (lo strumento di training associato a Kraken) per l'addestramento.

Normalizzazione del *ground truth* e creazione di *splits* e *charset* Una volta generato il file di *ground truth*, si procede alla sua validazione e normalizzazione, nonché alla creazione degli insiemi di training, validation e test e del relativo insieme di caratteri (*charset*) effettivamente presenti nei dati. Il passo successivo consiste nel suddividere i dati normalizzati in insiemi di training, validation e test e nell'estrarre il charset effettivamente utilizzato.

Come verifica finale di questa fase, è stato controllato che il file prodotto fosse coerente con le attese.

Filtraggio delle immagini per la lettura con `ketos` Per facilitare la lettura delle immagini da parte di `ketos`, le immagini delle righe sono state ulteriormente normalizzate con `ImageMagick`. Una volta ottenute le immagini nel formato desiderato, i percorsi all'interno dei file di *ground truth* sono stati aggiornati.

Mappatura delle immagini Per comodità, sono stati poi estratti i soli percorsi delle immagini, da utilizzare come input diretto per `ketos`.

Creazione dei file *sidecar* compatibili con `ketos` Poiché le versioni più recenti di Kraken delegano a `ketos` la gestione del training, è stato necessario generare, per ogni immagine, un file `.gt.txt` contenente la trascrizione corrispondente.

3 Risultati sperimentali

In questa sezione vengono riportati i risultati ottenuti dal modello fine-tuned in diversi scenari di test, con l'obiettivo di valutare sia la capacità del modello di adattarsi alla specifica calligrafia su cui è stato addestrato, sia il suo grado di generalizzazione a scritture differenti.

3.1 Metrica utilizzata

Dato che il modello opera riga per riga producendo sequenze di caratteri, la valutazione naturale sarebbe basata su indicatori quali il *Character Error Rate* (CER) o il *Word Error Rate* (WER). Nel presente progetto, tuttavia, la metrica più rilevante per l'utente finale (l'operatore che utilizza le trascrizioni per la ricerca documentale e per adempiere agli obblighi normativi) è la correttezza delle singole parole.

Per questo motivo si considera la seguente metrica intuitiva:

$$\text{Accuratezza parole} = \frac{\# \text{ parole riconosciute correttamente}}{\# \text{ parole totali}}.$$

In tutti gli esperimenti ogni prova consiste nel sottoporre al modello un foglio contenente 100 parole manoscritte, e nel contare quante di esse vengono riconosciute correttamente. I valori riportati di seguito sono calcolati su 20 prove indipendenti per ciascuno scenario e vengono sintetizzati tramite media e deviazione standard.

Si segnala inoltre che i test condotti hanno permesso di riutilizzare parte dei documenti di verifica, poiché nella fase di test è stata disabilitata l'opzione di usare parte del campione riconosciuto come ulteriore materiale di addestramento.

3.2 Scenario 1: pagine usate in addestramento

Nel primo scenario il modello è stato testato su pagine appartenenti agli stessi documenti utilizzati per il training, quindi:

- stessa persona che ha scritto a mano tutte le pagine;
- stessa tipologia di carta e impaginazione;
- condizioni di scansione controllate (600 dpi e pre-elaborazione con ScanTailor).

Sono state condotte 20 prove indipendenti. In ciascuna prova il modello ha riconosciuto un foglio contenente 100 parole, e per ogni foglio è stato contato il numero di parole trascritte correttamente. La media sulle 20 prove risulta pari a:

$$\text{Accuratezza media} \approx 88\% \pm 2\%,$$

dove $\pm 2\%$ rappresenta approssimativamente la deviazione standard tra le prove (cioè, nella maggior parte dei casi, l'accuratezza per singolo foglio ricade nell'intervallo 86–90%).

Gli errori non comportano quasi mai uno stravolgimento completo della parola, ma consistono piuttosto in sostituzioni di singole lettere graficamente simili (ad esempio “*casa*” riconosciuta come “*caso*”). Questi risultati indicano che il *fine tuning* ha effettivamente permesso al modello di apprendere in maniera soddisfacente la calligrafia specifica dell'autore dei documenti.

3.3 Scenario 2: altra persona, stessa tipologia di fogli

Nel secondo scenario l'OCR è stato applicato a pagine manoscritte da una persona diversa, ma su fogli con caratteristiche simili (righe prestampate, impaginazione analoga a quella dei documenti usati in addestramento).

Anche in questo caso sono state condotte 20 prove indipendenti, ciascuna su un foglio contenente 100 parole. La media sulle 20 prove risulta pari a:

$$\text{Accuratezza media} \approx 61\% \pm 2\%,$$

cioè il modello riconosce correttamente in media 61 parole su 100, con una variabilità tra le prove di circa due punti percentuali.

L'output rimane comunque leggibile e utile come bozza iniziale, ma la quantità di correzioni manuali richieste aumenta sensibilmente. Questa diminuzione di prestazioni mette in luce un aspetto atteso: con un dataset di addestramento relativamente limitato (circa 150 pagine), il modello tende a specializzarsi sulla calligrafia di riferimento e fatica a generalizzare a scritture diverse, pur mantenendo una certa capacità di riconoscere la struttura delle parole.

3.4 Ruolo della qualità di scansione

Durante i test è emerso che una parte consistente delle difficoltà non è dovuta soltanto alla calligrafia, ma anche alla qualità della scansione. Pagine acquisite con inclinazione eccessiva, contrasto basso o sfocatura locale risultano notevolmente più difficili da riconoscere, indipendentemente dal contenuto testuale.

Ciò suggerisce che, oltre al miglioramento del modello, un elemento cruciale per aumentare l'accuratezza complessiva è il controllo del processo di acquisizione (scelta dello scanner, impostazioni stabili, eventuale normalizzazione delle immagini prima del riconoscimento).

3.5 Sintesi dei risultati

La Tabella 1 riassume i risultati principali:

Scenario di test	Accuratezza parole (media $\pm$ dev. std.)
Stessa persona (pagine dei documenti di riferimento)	88% $\pm$ 2%
Altra persona, stessa tipologia di fogli	61% $\pm$ 2%

Tabella 1: Accuratezza a livello di parola in due scenari di test, calcolata su 20 prove indipendenti per scenario. Ogni prova prevede il riconoscimento di un foglio contenente 100 parole.

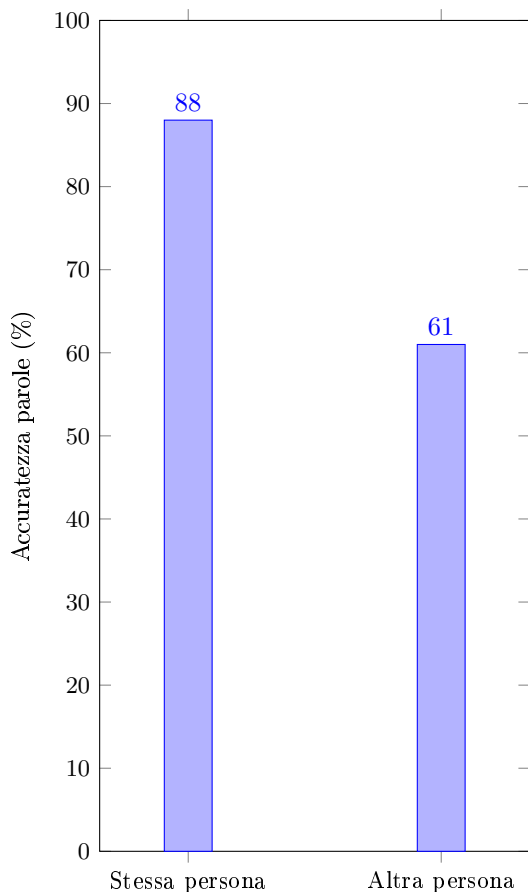


Figura 1: Accuratezza media a livello di parola nei due scenari di test considerati (20 prove da 100 parole ciascuna per scenario).

Dal punto di vista applicativo, questi risultati mostrano che il modello fine-tuned, pur non essendo ancora utilizzabile in modo completamente automatico, è già in grado di ridurre in misura significativa il carico di trascrizione manuale per la calligrafia su cui è stato addestrato. Inoltre, i test su una seconda persona confermano che, con un aumento del numero di pagine annotate e un maggiore controllo sulla qualità di scansione, è realistico puntare a livelli di accura-

tezza più elevati anche per popolazioni di scriventi più ampie.

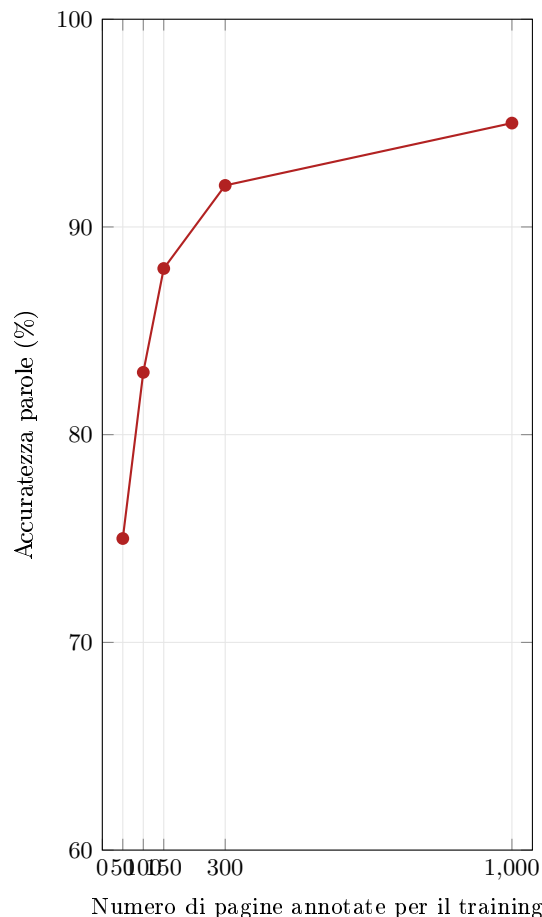


Figura 2: Andamento qualitativo atteso dell'accuratezza a livello di parola al crescere del numero di pagine annotate utilizzate per il training. Il punto a 150 pagine corrisponde ai risultati effettivamente ottenuti; gli altri valori sono stime indicative.

4 Conclusioni

I risultati ottenuti indicano che il modello fine-tuned rappresenta una prima base operativa per l'introduzione di un OCR per manoscritti italiani all'interno dell'INRCA. Nel caso della calligrafia specifica utilizzata per l'addestramento, il sistema raggiunge un'accuratezza a livello di parola prossima al 90%, consentendo di ridurre in modo sensibile il tempo necessario per produrre trascrizioni utilizzabili.

Tuttavia, il modello non è ancora sufficientemente robusto da poter essere impiegato in maniera completamente automatica sulla documentazione clinica reale. In particolare:

- la generalizzazione a scritture di persone diverse risulta ancora limitata (circa il 61% di parole corrette, con una deviazione standard di 2%);
- la qualità della scansione influisce in maniera significativa sui risultati;
- permane la necessità di una supervisione umana per la correzione degli errori.

Nonostante questi limiti, il lavoro dimostra la fattibilità tecnica di uno strumento OCR specializzato, eseguibile interamente in locale e potenzialmente integrabile nei flussi documentali dell'istituto. Ciò costituisce un elemento importante per motivare la richiesta di fondi dedicati allo sviluppo di una soluzione più completa e matura.

4.1 Sviluppi futuri

Per rendere il sistema sufficientemente affidabile in ambito produttivo saranno necessari ulteriori passaggi:

- aumentare significativamente il numero di pagine annotate, passando dalle circa 150 attuali ad almeno un ordine di grandezza superiore (nell'ordine delle mille pagine), in modo da migliorare la capacità di generalizzazione del modello;
- coinvolgere il DPO aziendale per definire un protocollo di selezione, anonimizzazione e utilizzo di un campione rappresentativo di documenti clinici reali;
- rafforzare la pipeline di acquisizione delle immagini, standardizzando il processo di scansione e le operazioni di pre-elaborazione;
- valutare l'introduzione di un modulo di post-processing linguistico (ad esempio un correttore contestuale) che possa correggere automaticamente gli errori più frequenti sulla base del lessico e del contesto.

Il progetto presentato è stato giudicato di interesse nazionale dal mio dirigente di comparto, aprendo la possibilità di richiedere risorse dedicate, inclusa eventualmente l'assunzione a tempo determinato di personale con il compito specifico di preparare i dati di addestramento. In questa prospettiva, il lavoro svolto costituisce un primo tassello concreto verso la realizzazione di uno strumento OCR conforme sia alle esigenze operative sia agli obblighi normativi in materia di protezione dei dati e conservazione della documentazione storica.

Riferimenti bibliografici

- [1] *Kraken OCR – repository GitHub*. <https://github.com/mittagessen/kraken>.
- [2] *Kraken OCR – sito ufficiale*. <https://kraken.re/main/index.html>.
- [3] *OCR models community (Zenodo)*. https://zenodo.org/communities/ocr_models/records?q=kraken&l=list&p=1&s=10&sort=bestmatch.